COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	7/27/2026	**DATE SUBMITTED:**	7/28/2026
NAME:	Alvarez, Joshua Caleb P.	**INSTRUCTOR:**	Engr.Jimlord Quejado

ACTIVITY NO. Hoa 3.1 [ACTIVITY NAME] HANDS ON 3.1 Linked Lists

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++
A.1. Singly Linked List

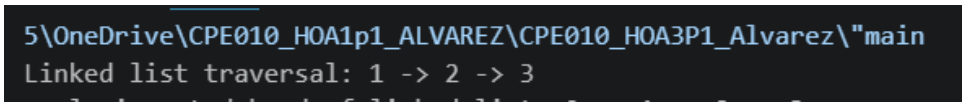
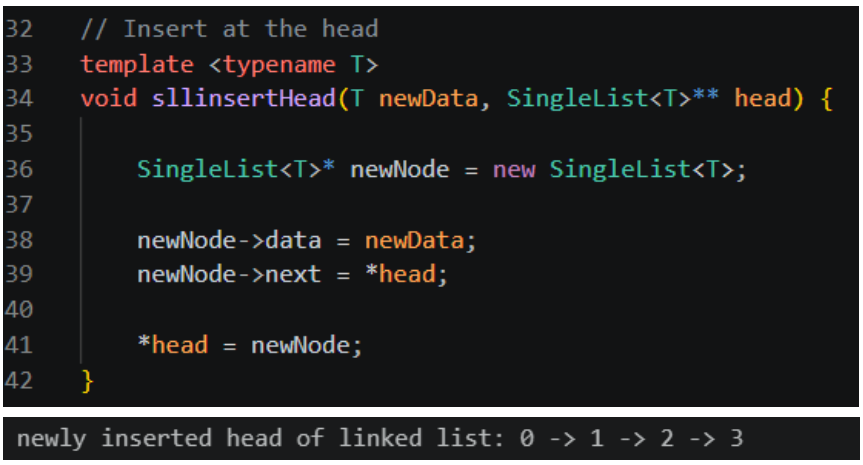
Screenshot	
Discussion	The output shows that the linked list was successfully created and traversed from the head node to the last node. Each node is connected using pointers, allowing the program to display the elements in the correct order. This simple implementation helped me understand how linked lists work and how nodes are linked together. It also showed the importance of pointers in accessing and managing data. The program can be improved by adding functions for insertion, deletion, and other operations to make the linked list more flexible and easier to use.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at Head	

```
// Traverse the list
template <typename T>
void listTraversal(SingleList<T>* head) {

    while (head != nullptr) {

        std::cout << head->data;

        if (head->next != nullptr) {
            std::cout << " -> ";
        }

        head = head->next;
    }

    std::cout << std::endl;
}
```

```
5\OneDrive\CPE010_HOA1p1_ALVAREZ\CPE010_HOA3P1_Alvarez\"main
Linked list traversal: 1 -> 2 -> 3
```

```
32   // Insert at the head
33   template <typename T>
34   void sllinsertHead(T newData, SingleList<T>** head) {
35
36       SingleList<T>* newNode = new SingleList<T>;
37
38       newNode->data = newData;
39       newNode->next = *head;
40
41       *head = newNode;
42   }
```

```
newly inserted head of linked list: 0 -> 1 -> 2 -> 3
```

Insertion at any part of the list

```cpp
// Insert after a specific node
template <typename T>
void sllGeneralinsertion(T newData, SingleList<T>* prevNode) {

    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null." << std::endl;
        return;
    }

    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}
```

```
Insertion at any part of the list: 0 -> 3 -> 4 -> 1 -> 2 -> 3
```

Insertion at the end

```cpp
// Insert at the end
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head) {

    // Make a new file
    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;
    newNode->next = nullptr;

    // Empty list
    if (*head == nullptr) {
        *head = newNode;
        return;
    }

    SingleList<T>* currentNode = *head;

    while (currentNode->next != nullptr) {
        currentNode = currentNode->next;
    }

    currentNode->next = newNode;
}
```

```
Insertion at the end: 0 -> 3 -> 4 -> 1 -> 2 -> 3 -> 10
```

Deletion of a node

```cpp
// Delete a node
template <typename T>
void sllDelete(T findData, SingleList<T>** head) {

    if (*head == nullptr)
        return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    // Search for the node
    while (currNode != nullptr && currNode->data != findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    // Not found
    if (currNode == nullptr)
        return;

    // Delete head
    if (prevNode == nullptr) {
        *head = currNode->next;
    }
    // Delete middle/end
    else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

```
117  // Delete entire list
118  template <typename T>
119  void sllDeleteList(SingleList<T>** head) {
120
121      SingleList<T>* currentNode = *head;
122
123      while (currentNode != nullptr) {
124
125          SingleList<T>* temp = currentNode;
126          currentNode = currentNode->next;
127
128          delete temp;
129      }
130
131      *head = nullptr;
132  }
133
134  #endif
```

```
deleting a node:
0 -> 3 -> 4 -> 1 -> 2 -> 3
```

Table 3-2. Code for the List Operations

A

Source Code

main.cpp > main()

```cpp
// Alvarez, Joshua Caleb P.
#include <iostream>
#include "singly_ll.h"

int main(){

    SingleList<int> *head =  new SingleList<int>;
    SingleList<int> *second = new SingleList<int>;
    SingleList<int> *third = new SingleList<int>;

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = nullptr;

    std::cout << "Linked list traversal: ";
    listTraversal(head);

```

Console

```
5\OneDrive\CPE010_HOA1p1_ALVAREZ\CPE010_HOA3P1_Alvarez\"main
Linked list traversal: 1 -> 2 -> 3
```

B

Source Code

```cpp
    std::cout<<"newly inserted head of linked list: ";
    sllinsertHead(0,&head);
    listTraversal(head);
```

Console

```
newly inserted head of linked list: 0 -> 1 -> 2 -> 3
```

C

Source Code

```cpp
    std::cout <<"newly inserted after a node of linked list: ";
    sllGeneralinsertion(3,head);
    listTraversal(head);
```

Console

```
newly inserted after a node of linked list: 0 -> 3 -> 1 -> 2 -> 3
```

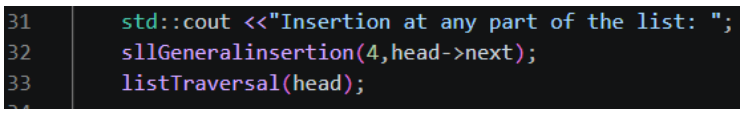
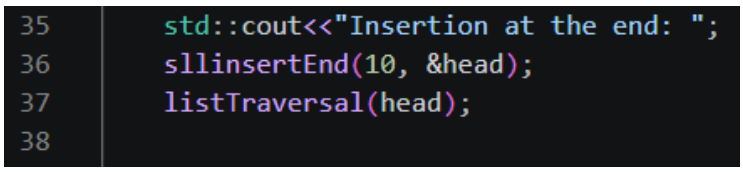
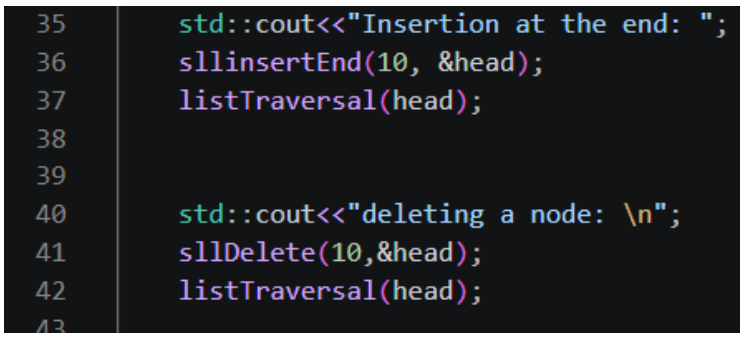
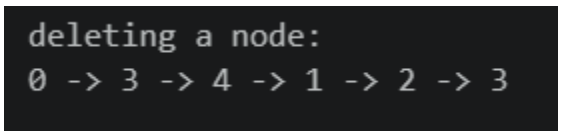
D	Source Code	
	Console	
E	Source Code	
	Console	
F	Source Code	
	Console	

Table 3-3. Code and Analysis for Singly Linked Lists

A.2. Doubly Linked List

Screenshots

```cpp
#ifndef DOUBLE_LL_H
#define DOUBLE_LL_H

#include <iostream>

template <typename T>
class doubleList {
public:
    T data;
    doubleList<T>* next = nullptr;
    doubleList<T>* prev = nullptr;
};
template <typename T>
doubleList<T>* new_Node(T newData) {

    // Create a new node
    doubleList<T>* newNode = new doubleList<T>;

    // Store the data in the node
    newNode->data = newData;

    // Initialize both pointers
    newNode->next = nullptr;
    newNode->prev = nullptr;

    // Return the newly created node
    return newNode;
}
```

```cpp
std::cout << "Initial Traversal" << std::endl;
dllTraverse(head);

// Insert at the head
std::cout << "\nInsertion at Head" << std::endl;
dllInsertHead('A', &head);
dllTraverse(head);
```

Analysis

1. Traversal (`dllTraverse()`)

The `dllTraverse()` function is used to display all the nodes in the doubly linked list. It first starts from the head node and follows the `next` pointer until it reaches the last node, printing each data value along the way. After reaching the end of the list, the function uses the `prev` pointer to move backward and display the nodes in reverse order. The output shows that both the `next` and `prev` pointers are working correctly because the list can be traversed in both directions without any missing or incorrect nodes. This function helped me understand how a doubly linked list differs from a singly linked list and how using two pointers allows easier navigation through the list.

```cpp
#include <iostream>
#include "doubly_ll.h"

int main() {

    // Create the head node
    doubleList<char>* head = new_Node('C');

    // Create the remaining nodes
    doubleList<char>* second = new_Node('P');
    doubleList<char>* third = new_Node('E');

```

```cpp
    std::cout << "Initial Traversal" << std::endl;
    dllTraverse(head);

```

```
Initial Traversal
Forward Traversal
C P E
```

```cpp
    // Insert at the head
    std::cout << "\nInsertion at Head" << std::endl;
    dllInsertHead('A', &head);
    dllTraverse(head);

```

```cpp
    std::cout << std::endl;

    // Display the list in reverse order
    std::cout << "Reverse Traversal" << std::endl;

    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
```

```
Reverse Traversal
E P C
```

2.Insertion at Head (`dllInsertHead()`)

The `dllInsertHead()` function inserts a new node at the beginning of the doubly linked list. After creating the new node, it updates the `next` pointer of the new node to point to the old head and changes the `prev` pointer of the old head to point back to the new node. The head pointer is then updated to the new node. The output shows that the new node becomes the first element in the list while keeping all the connections between the nodes correct. This operation demonstrates how both pointers must be updated when inserting a node at the beginning of a doubly linked list.

```cpp
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead) {

    // Create a new node
    doubleList<T>* newNode = new_Node(newData);

    // If the list is empty, make the new node the head
    if (*currentHead == nullptr) {
        *currentHead = newNode;
        return;
    }

    // Link the new node to the current head
    newNode->next = *currentHead;

    // Update the previous pointer of the old head
    (*currentHead)->prev = newNode;

    // Make the new node the new head
    *currentHead = newNode;
}

```

```cpp
std::cout << "Initial Traversal" << std::endl;
dllTraverse(head);

// Insert at the head
std::cout << "\nInsertion at Head" << std::endl;
dllInsertHead('A', &head);
dllTraverse(head);

// Insert at position 3
std::cout << "\nInsertion at Position 3" << std::endl;
dllInsertPosition('G', 3, &head);
dllTraverse(head);

// Insert at the end
std::cout << "\nInsertion at End" << std::endl;
dllInsertEnd('Z', &head);
dllTraverse(head);
```

```
Insertion at Head
Forward Traversal
A C P E
Reverse Traversal
E P C A
```

3. Insertion at Any Position (`dllInsertPosition()`)

The `dllInsertPosition()` function adds a new node at a specified position in the doubly linked list. The function first checks if the given position is valid before moving to the correct location. It then updates both the `next` and `prev` pointers of the surrounding nodes so that the new node is connected properly. The output confirms that the node is inserted in the correct position without affecting the other nodes. This operation shows the importance of updating both pointers to maintain the structure of the doubly linked list.

```cpp
template <typename T>
void dllInsertEnd(T newData, doubleList<T>** currentHead) {

    // Create a new node
    doubleList<T>* newNode = new_Node(newData);

    // If the list is empty, insert the node as the head
    if (*currentHead == nullptr) {
        *currentHead = newNode;
        return;
    }

    // Traverse to the last node
    doubleList<T>* temp = *currentHead;

    while (temp->next != nullptr) {
        temp = temp->next;
    }

    // Connect the new node at the end
    temp->next = newNode;
    newNode->prev = temp;
}
```

```cpp
    // Insert at the end
    std::cout << "\nInsertion at End" << std::endl;
    dllInsertEnd('Z', &head);
    dllTraverse(head);
```

```
Insertion at End
Forward Traversal
A C G P E Z
Reverse Traversal
Z E P G C A
```

Insertion at End (`dllInsertEnd()`)

The `dllInsertEnd()` function inserts a new node at the end of the doubly linked list. It first traverses the list until it reaches the last node and then connects the new node by updating the `next` pointer of the last node and the `prev` pointer of the new node. The output shows that the new node is successfully added at the end of the list while keeping all existing nodes connected. This function demonstrates how new elements can be added without changing the earlier nodes in the list.

```cpp
template <typename T>
void dllDeletePosition(int position, doubleList<T>** currentHead) {

    // Return if the list is empty
    if (*currentHead == nullptr) {
        std::cout << "List is Empty!" << std::endl;
        return;
    }

    // Check if the position is valid
    if (position < 1) {
        std::cout << "Invalid Position!" << std::endl;
        return;
    }

    // Start from the head node
    doubleList<T>* temp = *currentHead;
```

5. Deletion (`dllDeletePosition()`)

The `dllDeletePosition()` function removes a node from a specified position in the doubly linked list. After locating the node to be deleted, it updates the `next` and `prev` pointers of the neighboring nodes to reconnect the list. The selected node is then removed from memory, and the remaining nodes stay properly linked. The output confirms that the correct node was deleted and that the list continues to work normally in both forward and reverse traversal. This operation helped me understand the importance of updating both pointers when removing a node from a doubly

```
    // Start from the head node
    doubleList<T>* temp = *currentHead;

    // Delete the first node
    if (position == 1) {
        *currentHead = temp->next;

        if (*currentHead != nullptr) {
            (*currentHead)->prev = nullptr;
        }

        delete temp;
        return;
    }
```

```
    // Traverse to the target position
    for (int i = 1; i < position && temp != nullptr; i
        temp = temp->next;
    }

    // Stop if the position is invalid
    if (temp == nullptr) {
        std::cout << "Invalid Position!" << std::endl;
        return;
    }

    // Connect the previous node to the next node
    if (temp->prev != nullptr) {
        temp->prev->next = temp->next;
    }

    // Update the previous pointer of the next node
    if (temp->next != nullptr) {
        temp->next->prev = temp->prev;
    }
```

```
    // Delete node at position 3
    std::cout << "\nDeletion at Position 3" << std::endl;
    dllDeletePosition(3, &head);
    dllTraverse(head);

    // Free all allocated memory
    dllDestroy(&head);

    return 0;
}
```

```
Deletion at Position 3
Forward Traversal
A C P E Z
Reverse Traversal
Z E P C A
```

linked list.

Table 3-4. Modified Operations for Doubly Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing
Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple Players take a turn one by one in a circle.
These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.
We'll take the use case of a music player. It should have following functions supported:

Create a playlist using multiple songs.
Add songs to the playlist.
Remove a song from the playlist.
Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:
Design the basic structure that supports circular data representation.
After that, implement the insert and delete functions in the structure.
Implement a function for traversing the playlist.
The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

Provide the following:

Summary of lessons learned

Analysis of the procedure

Analysis of the supplementary activity

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas

for improvement?

SUPPLEMENTARY ACTIVITY CODE

Circular_ll.h

```h
#ifndef CIRCULAR_LL_H
#define CIRCULAR_LL_H

#include <iostream>
#include <string>

template <typename T>
class CircularList {
public:
    T song;
    CircularList<T>* next = nullptr;
};

// Add a song to the playlist
template <typename T>
void insertSong(T newSong, CircularList<T>** head) {

    CircularList<T>* newNode = new CircularList<T>;
    newNode->song = newSong;

    if (*head == nullptr) {
        *head = newNode;
        newNode->next = newNode;
        return;
    }
```

```
26
27      CircularList<T>* temp = *head;
28
29      while (temp->next != *head) {
30          temp = temp->next;
31      }
32
33      temp->next = newNode;
34      newNode->next = *head;
35  }
36
37  // Remove a song from the playlist
38  template <typename T>
39  void removeSong(T targetSong, CircularList<T>** head) {
40
41      if (*head == nullptr) {
42          return;
43      }
44
45      CircularList<T>* current = *head;
46      CircularList<T>* previous = nullptr;
47
48  do {
49
50      if (current->song == targetSong) {
```

```
50      if (current->song == targetSong) {
52          if (previous == nullptr) {
53
54              if (current->next == *head) {
55                  delete current;
56                  *head = nullptr;
57                  return;
58              }
59
60              CircularList<T>* last = *head;
61
62              while (last->next != *head) {
63                  last = last->next;
64              }
65
66              *head = current->next;
67              last->next = *head;
68          }
69
70          else {
71              previous->next = current->next;
72          }
73
74          delete current;
75          return;
```

```
78              previous = current;
79              current = current->next;
80
81          } while (current != *head);
82
83          std::cout << "Song not found." << std::endl;
84      }
85
86      // Play all songs
87      template <typename T>
88      void playSongs(CircularList<T>* head) {
89
90          if (head == nullptr) {
91              std::cout << "Playlist is empty." << std::endl;
92              return;
93          }
94
95          CircularList<T>* current = head;
96
97          std::cout << "Now Playing Playlist:" << std::endl;
98
99          do {
100             std::cout << current->song << std::endl;
101             current = current->next;
```

```
    } while (current != head);
}

// Next song
template <typename T>
CircularList<T>* nextSong(CircularList<T>* current) {

    if (current != nullptr)
        return current->next;

    return nullptr;
}

// Previous song
template <typename T>
CircularList<T>* previousSong(CircularList<T>* head, CircularList<T>* current) {

    if (head == nullptr || current == nullptr)
        return nullptr;

    CircularList<T>* temp = head;

    while (temp->next != current) {
        temp = temp->next;
    }

    return temp;
}

#endif
```

Short Analysis

The `circular_ll.h` file contains the functions needed to manage the music playlist using a circular linked list. It includes functions for adding songs, removing songs, playing all songs, and moving to the next or previous song. Since the last node points back to the first node, the playlist can continue looping without reaching the end. This implementation helped me understand how circular linked lists work and how they can be applied to real-world applications such as a music player.

Main.cpp

```cpp
#include <iostream>
#include <string>
#include "circular_ll.h"

int main() {

    CircularList<std::string>* playlist = nullptr;

    // Create the playlist
    insertSong(std::string("Blue"), &playlist);
    insertSong(std::string("Yellow"), &playlist);
    insertSong(std::string("Perfect"), &playlist);

    std::cout << "=== Original Playlist ===\n";
    playSongs(playlist);

    // Add a new song
    std::cout << "\n=== Add Song ===\n";
    insertSong(std::string("Photograph"), &playlist);
    playSongs(playlist);

    // Remove a song
    std::cout << "\n=== Remove Song ===\n";
    removeSong(std::string("Yellow"), &playlist);
    playSongs(playlist);

```

```cpp
    // Demonstrate next and previous
    CircularList<std::string>* current = playlist;

    std::cout << "\n=== Music Player ===\n";

    std::cout << "Current Song : " << current->song << std::endl;

    current = nextSong(current);
    std::cout << "Next Song    : " << current->song << std::endl;

    current = nextSong(current);
    std::cout << "Next Song    : " << current->song << std::endl;

    current = previousSong(playlist, current);
    std::cout << "Previous Song: " << current->song << std::endl;

    current = previousSong(playlist, current);
    std::cout << "Previous Song: " << current->song << std::endl;

    return 0;
}
```

Short Analysis

The `main.cpp` file was used to test the different operations of the circular linked list. It first created a playlist by adding several songs, then displayed the playlist using the play function. After that, a new song was added and another song was removed to check if the playlist was updated correctly. The program also demonstrated the next and previous song functions to show how a circular linked list allows continuous navigation through the playlist. Overall, the driver program confirmed that all the required operations were working properly.

CODE OUTPUT

```
=== Original Playlist ===
Now Playing Playlist:
Blue
Yellow
Perfect
```

```
=== Add Song ===
Now Playing Playlist:
Blue
Yellow
Perfect
Photograph
```

```
=== Remove Song ===
Now Playing Playlist:
Blue
Perfect
Photograph
```

```
=== Music Playlist ===
Current Song : Blue
Next Song    : Perfect
Next Song    : Photograph
Previous Song: Perfect
Previous Song: Blue
```

Short Analysis

The output shows that the playlist was created successfully and that all the songs were displayed in the correct order. After inserting a new song, the playlist was updated to include the new entry. When a song was removed, the playlist displayed the remaining songs correctly, showing that the links between the nodes were updated properly. The next and previous functions also worked as expected by moving through the playlist without stopping, demonstrating the circular structure of the linked list.

Conclusion

This supplementary activity helped me understand how a circular linked list can be used in a real-life application such as a music playlist. Unlike a regular linked list, the last node is connected back to the first node, allowing the songs to be played continuously. While creating the program, I learned how to insert and remove songs while maintaining the circular structure of the list. I also understood how the next and previous functions work by moving through the playlist without reaching an end.Despite from that I become mentally challenged since this code is full of analyzing and most of the time I'm drained to write a code that keeps the code wrong or keeps error at the same time. Lastly after this activity it actually improved my understanding of linked lists and how they can be applied in different programs.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement.

I think I did well in this activity because I was able to complete the required operations and understand how a circular linked list works. Although I encountered a few errors while writing the code and also became mentally drained, I was able to fix them and make the program work correctly with the use of Ai tools. This activity improved my problem-solving skills and gave me more confidence in using linked lists. However, I still need more practice in handling this kind of cases and writing cleaner, more efficient code. With more experience, I believe I can develop more complex data structure programs with fewer mistakes.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chat gpt and Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarization of analysis and reflections, assistance in explaining code, debugging code errors, formatting the report, and improving the clarity of written responses.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **To help me find the errors and how to fix and also Ai help me what areas need to improve of my code, explanation of analysis, Ai helps me downloading or install the mw64 to my pc, and also helps me where i put screenshots**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.